General Information

Hardware Specs:

- Processor 11th Gen Intel(R) Core(TM) i5-11300H @ 3.10GHz, 3110 Mhz, 4 Core(s), 8 Logical Processor(s)
- 16.0 GB Ram
- Windows 11

Library Versions:

- numpy=2.3.5
- numba=0.63.1
- Other specifics can be found in "environment.yml"

Measurement Methodology:

- Warmup: A single warmup run was used in the numba implementation
- Trials: Manual time captures (the ones excluding profiling) was captured over 3 trials.

cProfile Measurements (Milestone 1)

cProfile Naive Solution

```
Thu Feb 26 14:15:34 2026    naive_profile.prof

         23877158 function calls in 7.985 seconds

   Ordered by: cumulative time
   List reduced from 40 to 10 due to restriction <10>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    7.985    7.985 {built-in method builtins.exec}
        1    0.002    0.002    7.985    7.985 <string>:1(<module>)
        1    0.496    0.496    7.983    7.983 C:\...\NumericalScientificComputing\Mandelbrot\mandelbrot_naive.py:42(compute_mandelbrot_naive)
  1048576    6.290    0.000    7.485    0.000 C:\...\NumericalScientificComputing\Mandelbrot\mandelbrot_naive.py:24(mandelbrot_point)
 22828510    1.194    0.000    1.194    0.000 {built-in method builtins.abs}
        1    0.000    0.000    0.003    0.003 C:\...\miniforge3\envs\nsc2026\Lib\site-packages\numpy\lib\_function_base_impl.py:5134(meshgrid)
        3    0.000    0.000    0.003    0.001 C:\...\miniforge3\envs\nsc2026\Lib\site-
packages\numpy\lib\_function_base_impl.py:5280(<genexpr>)
        2    0.003    0.001    0.003    0.001 {method 'copy' of 'numpy.ndarray' objects}
        2    0.000    0.000    0.000    0.000 C:\Users\chris\miniforge3\envs\nsc2026\Lib\site-
packages\numpy\_core\function_base.py:26(linspace)
        1    0.000    0.000    0.000    0.000 C:\...\miniforge3\envs\nsc2026\Lib\site-
packages\numpy\lib\_stride_tricks_impl.py:481(broadcast_arrays)
```

cProfile Naive Solution

```
Thu Feb 26 14:15:35 2026    vectorized_profile.prof

         77 function calls in 0.983 seconds
   Ordered by: cumulative time
   List reduced from 42 to 10 due to restriction <10>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.983    0.983 {built-in method builtins.exec}
        1    0.002    0.002    0.983    0.983 <string>:1(<module>)
        1    0.975    0.975    0.981    0.981
C:\...\NumericalScientificComputing\Mandelbrot\mandelbrot_vectorized.py:24(compute_mandelbrot_vectorized)
        1    0.003    0.003    0.003    0.003 C:\Users\chris\miniforge3\envs\nsc2026\Lib\site-packages\numpy\_core\numeric.py:98(zeros_like)
        1    0.000    0.000    0.003    0.003 C:\...\miniforge3\envs\nsc2026\Lib\site-packages\numpy\lib\_function_base_impl.py:5134(meshgrid)
        3    0.000    0.000    0.003    0.001 C:\...\miniforge3\envs\nsc2026\Lib\site-
packages\numpy\lib\_function_base_impl.py:5280(<genexpr>)
        2    0.002    0.001    0.002    0.001 {method 'copy' of 'numpy.ndarray' objects}
        2    0.000    0.000    0.000    0.000 C:\Users\chris\miniforge3\envs\nsc2026\Lib\site-
packages\numpy\_core\function_base.py:26(linspace)
        1    0.000    0.000    0.000    0.000 C:\...\miniforge3\envs\nsc2026\Lib\site-
packages\numpy\lib\_stride_tricks_impl.py:481(broadcast_arrays)
        1    0.000    0.000    0.000    0.000 C:\...\miniforge3\envs\nsc2026\Lib\site-
packages\numpy\lib\_stride_tricks_impl.py:546(<listcomp>)
```

Questions regarding cProfile

Which function takes most total time:

For the naive method, the 2 functions that dominated in terms of time was the builtin.abs function when evaluating each point at for the threshold (1.194s), and the 'mandelbrot_point' computing function, which dominated with 6.290s.

In the vectorized/Numpy implementation, it was pretty much only the vectorized computations that contributed to the total time, specifically 'compute_mandelbrot_vectorized' with 0.975s out of the 0.983s.

Are there functions called surprisingly many times:

There weren't any suprising results for the vectorized/numpy implementation, most calls were not in the top 10 time contributors. However, the naive implementation had significantly many more calls; the mandelbrot_point function was called 1048576 times, and the builtin.abs function was called 22828510 times.

How does the Numpy compare to the naive:

The Numpy implementation is way faster than the naive one, and performs way less function calls which probably also eliminates some overhead.

Deep Profiling (Milestone 2)

Results

```
Wrote profile results to 'mandelbrot_naive.py.lprof'
Timer unit: 1e-06 s

Total time: 13.8561 s
File: mandelbrot_naive.py
Function: mandelbrot_point at line 25

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
    25                                           @profile
    26                                           def mandelbrot_point(complex_input: complex, max_iterations: int = 1000):
    27                                               """
    28                                               Description:
    29                                               - Takes a single complex number c
    30                                               - returns iteration count
    31                                               - test with known points (e.g., c=0 should be max_iterations)
    32                                               """
    33                                               # Initialize z to 0
    34   1048576     134971.4      0.1      1.0      z_0 = 0 + 0j
    35
    36  23008310    2651595.1      0.1     19.1      for iteration in range(max_iterations):
    37  22828510    5341363.9      0.2     38.5          if abs(z_0) > 2:
    38    868776     128416.2      0.1      0.9              return iteration
    39  21959734    5579249.2      0.3     40.3          z_0 = z_0**2 + complex_input
    40
    41                                               # Return the max iteartion count if the point does not escape
    42    179800      20511.7      0.1      0.1      return max_iterations
```

```
Total time: 19.7165 s
File: mandelbrot_naive.py
Function: compute_mandelbrot_naive at line 44

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
    44                                           @profile
    45                                           def compute_mandelbrot_naive(x_space, y_space, resolution, max_iterations):
    46                                               """
    47                                               x_space: Describes the range of the real axis (e.g., [-2.0, 1.0])
    48                                               y_space: Describes the range of the imaginary axis (e.g., [-1.5, 1.5])
    49                                               param resolution: Number of points along each axis (e.g., 1000)
    50
    51                                               Description:
    52                                               - Creates a grid of complex numbers representing the points in the complex plane
    53                                               - For each point in the grid, it calls mandelbrot_point to determine the iteration count
    54                                               - Returns a 2D array of iteration counts corresponding to each point in the grid
    55                                               """
    56
    57                                               # Create a grid of complex numbers representing the points in the complex plane
    58         1         69.9     69.9      0.0      x = np.linspace(x_space[0], x_space[1], resolution) # Real axis values
    59         1         16.7     16.7      0.0      y = np.linspace(y_space[0], y_space[1], resolution) # Imaginary axis values
    60         1       2581.7   2581.7      0.0      X, Y = np.meshgrid(x, y)
    61
    62         1       7892.7   7892.7      0.0      complex_grid = X + 1j * Y # Create a grid of complex numbers from [0,0] to [resolution-1,resolution-1]
    63
    64                                               # Initialize a 2D array to store the iteration counts
    65         1         15.3     15.3      0.0      iteration_counts = np.zeros(complex_grid.shape, dtype=int)
    66
    67      1025        117.5      0.1      0.0      for i in range(resolution):
    68   1049600     123823.6      0.1      0.6          for j in range(resolution):
    69   1048576     275749.7      0.3      1.4              complex_input = complex_grid[i, j]
    70   1048576   18999805.2     18.1     96.4              iteration_count = mandelbrot_point(complex_input, max_iterations)
    71
    72                                                       # Store the iteration count in a 2D array
    73   1048576     306441.9      0.3      1.6              iteration_counts[i, j] = iteration_count
    74
    75         1          0.1      0.1      0.0      return iteration_counts
```

Questions

How many functions appear in each profile? What does this difference tell you about where the work actually happens?

The naive approach consists of significantly more function calls, as each point is calculated individually and sequentially. This is slow, but also introduces a lot of overhead. The NumPy approach eliminates a bunch of this using a vectorized approach.

Which lines dominate runtime? What fraction of total is spent in the inner loop?

According to line_profiler, 96.4% of the total time is spent in the inner loop 'iteration_count = mandelbrot_point(complex_input, max_iterations)'. Within the inner loop, the lines that dominate runtime are:

- 'if abs(z_0) > 2:' takes up 38.5% of the time.
- 'z_0 = z_0**2 + complex_input' takes up 40.3% of the time.
- 'for iteration in range(max_iterations):' takes up 19.1% of the time.

Based on your profiling results: why is NumPy faster than the naive Python?

NumPy is faster because it avoids executing millions of operations inside Python loops. In the naive version, each pixel requires a Python function call and many interpreted operations, which introduces significant overhead. Profiling shows that most of the runtime is spent inside the inner loop and repeated calls to abs() and complex arithmetic. NumPy moves these computations into optimized C code, where loops run at compiled speed instead of interpreter speed. This drastically reduces overhead and improves performance.

What would you need to change to make the naive version faster?

To make the naive version faster, the inner loop must be optimized since it dominates nearly all runtime. The main improvement would be to reduce Python overhead by avoiding repeated function calls and complex number operations. Replacing complex numbers with separate real and imaginary variables and using zr*zr + zi*zi > 4 instead of abs() would also improve performance.

Numba njit - Speedup Table (Milestone 3)

Speedups compared to Naive approach:

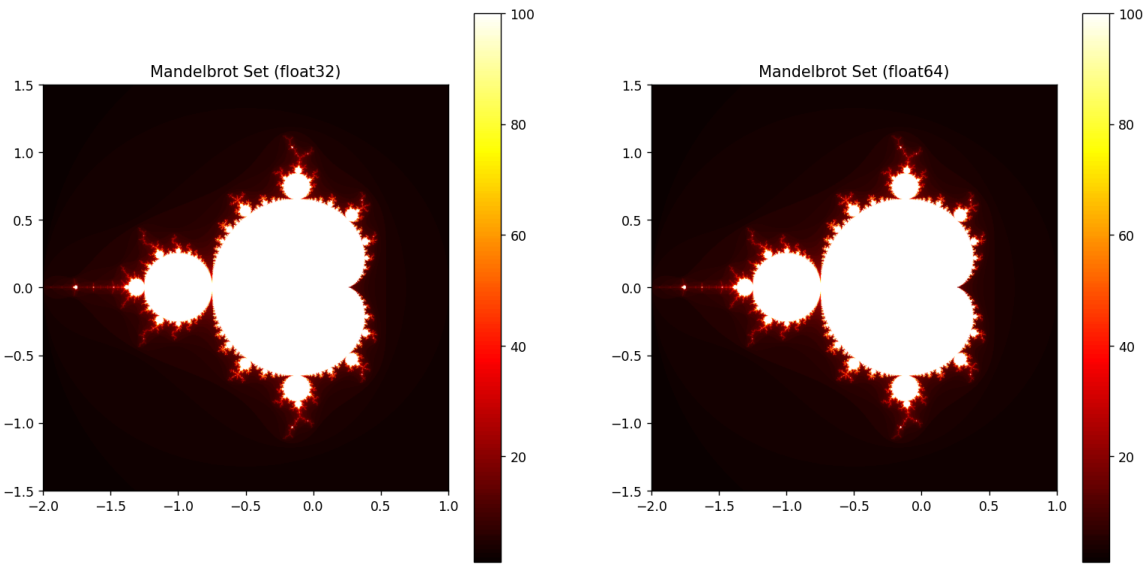
Approach	**Time**	**Speedup**
Naive approach	4.0889 seconds	1

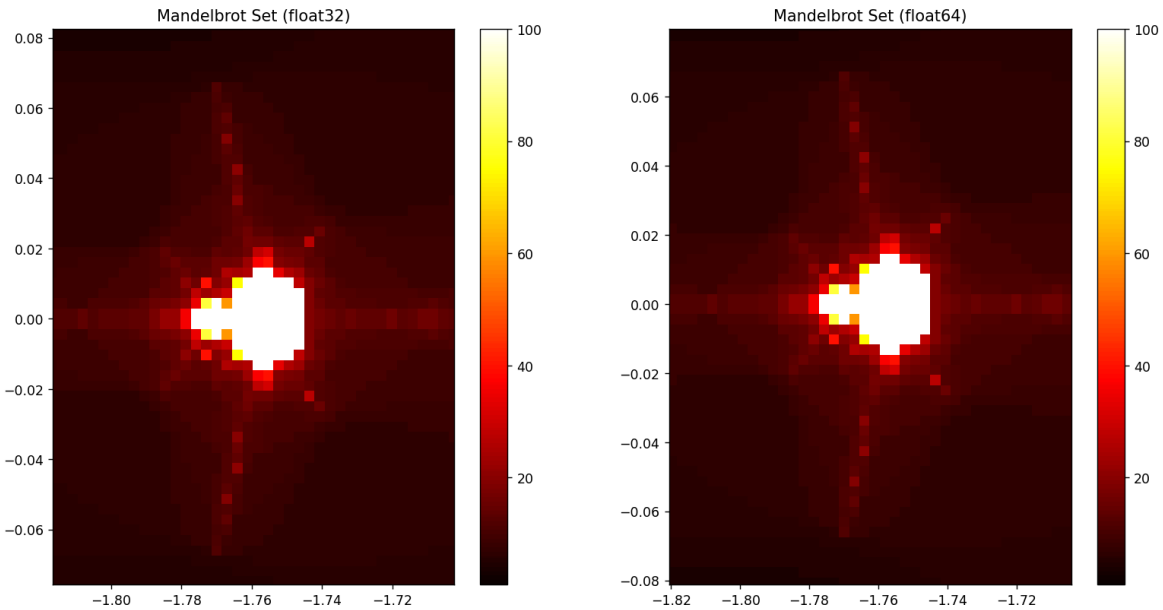
NumPy/vectorized	0.9811 seconds	4.168
Numba njit hybrid	0.428 seconds	9.55
Numba njit fully	0.062 seconds	65.95

Data Type Optimization (Milestone 4)

```shell
Running with precision: <class 'numpy.float32'>

Execution time: 0.071 seconds

Running with precision: <class 'numpy.float64'>

Execution time: 0.311 seconds
```

Inspecting the images, it can be seen that there is barely a difference between the quality. Because of this, it would make most sense to run a lower precision, e.g., float32, as it drastically improves computation time while barely sacrificing quality.




Mandelbrot Set (float32)
0.08
0.06
0.04
0.02
0.00
−0.02
−0.04
−0.06
−1.80
−1.78
−1.76
−1.74
−1.72
100
80
60
40
20
Mandelbrot Set (float64)
0.06
0.04
0.02
0.00
−0.02
−0.04
−0.06
−0.08
−1.82
−1.80
−1.78
−1.76
−1.74
−1.72
100
80
60
40
20